

Configuración de Entrenamiento Chatbot DianSistemas

Guía completa para entender, personalizar y reentrenar los modelos del chatbot.

Tabla de Contenidos

Visión general del pipeline de entrenamiento

1. Datos de entrenamiento (CSV)

- Campos del dataset

- Añadir nuevos ejemplos

- Comandos para reimportar datos

2. Intenciones y Etiquetas

- Intenciones configuradas (entrenamiento/datos/intenciones.csv)

- Etiquetas de entidades (entrenamiento/datos/etiquetas.csv)

3. Generación de datos .spacy

4. Configuración de spaCy (config.cfg)

- Hiperparámetros clave de entrenamiento

5. Entrenamiento del Modelo spaCy

- Métricas clave y objetivos

6. Configuración LLM (Ollama)

- Gestión de Modelfiles

7. Flujo Completo de Reentrenamiento

- Despliegue automatizado con Docker

8. Buenas Prácticas y Troubleshooting

- Promoción de ejemplos desde conversaciones reales

- Matriz de buenas prácticas

- Resolución de problemas (Troubleshooting)

Visión general del pipeline de entrenamiento

El sistema combina un procesamiento NLP estructurado mediante Django y spaCy con modelos LLM locales gestionados por Ollama. A continuación se describe el flujo integral de datos desde la captura inicial hasta la inferencia:

Etapa	Componente	Entrada / Salida	Función principal
1. Datos	CSV (nlp.csv)	Archivo plano tabular	Almacenamiento de frases etiquetadas con intención y entidades.
2. Persistencia	Django Models (EjemploNLP)	BD Relacional (PostgreSQL/SQLite)	Gestión, filtrado, versionado y promoción de ejemplos reales.
3. Compilación	.spacy (train/dev.spacy)	Binarios spaCy DocBin	Serialización optimizada de corpus de entrenamiento y evaluación.
4. Entrenamiento	spaCy Engine (config.cfg)	Modelo textcat + NER	Ajuste de pesos mediante Tok2Vec, TextCatBOW y vectores de es_core_news_lg.
5. Generación LLM	Ollama Modelfiles	Modelos locales (Llama 3.2)	Creación de prompts del sistema parametrizados por intención.

1. Datos de entrenamiento (CSV)

Archivo principal:

entrenamiento/datos/nlp.csv

El formato estructurado del archivo sigue el estándar delimitado por punto y coma (;):

texto;intencion;origen;entidades

Campos del dataset

Campo	Descripción	Ejemplo
texto	Texto del usuario (obligatorio)	"Quiero un presupuesto"
intencion	Categoría (compra, consulta_tecnica, contactar_humano, cerrar, confirmacion, otro)	compra

Campo	Descripción	Ejemplo
origen	Fuente: manual (humano), chatbot (auto), promovido	manual
entidades	Anotaciones ETIQUETA:texto separadas por	SERVICIO:auditoria EMPRESA:mi empresa

Añadir nuevos ejemplos

Existen dos métodos para registrar nuevos datos de entrenamiento:

- Edición directa del CSV: modificar el fichero en entrenamiento/datos/nlp.csv.
- Vía Django Shell ejecutando código interactivo:

```
from entrenamiento.models import EjemploNLP, Intencion
EjemploNLP.objects.create(
    texto='Nuevo ejemplo de compra',
    intencion=Intencion.objects.get(nombre='compra'),
    origen='manual'
)
```

Comandos para reimportar datos

```
# Limpiar y recargar todo desde cero
python manage.py cargar_nlp --limpiar
```

```
# Solo añadir nuevos registros (sin limpiar)
python manage.py cargar_nlp
```

2. Intenciones y Etiquetas

Intenciones configuradas (entrenamiento/datos/intenciones.csv)

Intención	Descripción	Estado
compra	Usuario quiere comprar / solicitar presupuesto / contratar	Activa
consulta_tecnica	Soporte técnico, errores de sistema, instalación	Activa
contactar_humano	Solicitud explícita de hablar con un agente humano	Activa
cerrar	Despedida, agradecimiento final, cierre de conversación	Activa
confirmacion	Aceptación de presupuestos o condiciones presentadas	Activa
otro	Saludos iniciales, información general, consultas fuera de dominio	Activa

Etiquetas de entidades (entrenamiento/datos/etiquetas.csv)

Etiqueta	Descripción
SERVICIO	Nombre del servicio, producto o módulo de software
EMPRESA	Nombre de la empresa u organización del cliente
PRECIO	Montos monetarios, cotizaciones, tarifas o costes
EMPLEADOS	Número de trabajadores o usuarios finales
SECTOR	Sector económico o actividad empresarial
CONTACTO	Dirección de email, teléfono o nombre de contacto
UBICACION	Dirección física, ciudad o provincia

3. Generación de datos .spacy

El comando de compilación transforma los modelos Django en datasets binarios spaCy:

```
python manage.py generar_nlp [--dev-ratio 0.2]
```

Proceso interno de compilación:

1. Lectura de todos los registros EjemploNLP activos en la base de datos.
2. Construcción de objetos Doc con textcat (exclusive_classes=True) y spans de NER.
3. Particionado determinístico train/dev basado en el hash unívoco del texto.
4. Escritura de los ficheros train.spacy y dev.spacy en entrenamiento/spacy/.

4. Configuración de spaCy (config.cfg)

El pipeline se define en entrenamiento/spacy/config.cfg combinando Tok2Vec y TextCatEnsemble:

```
[nlp]
lang = "es"
pipeline = ["tok2vec", "textcat"]
batch_size = 1000

[components.textcat]
factory = "textcat"
exclusive_classes = true

[paths]
vectors = "es_core_news_lg" # Vectores de 500k palabras
```

Hiperparámetros clave de entrenamiento

Parámetro	Valor	Descripción
max_steps	20000	Pasos máximos (detención temprana por patience)

Parámetro	Valor	Descripción
patience	1600	Pasos sin mejora en validación antes de parar
eval_frequency	200	Frecuencia de evaluación frente al dev set
dropout	0.1	Tasa de regularización por dropout
learn_rate	0.001	Tasa de aprendizaje (optimizador Adam)
batch_size	100 → 1000	Tamaño de batch dinámico (compounding)

5. Entrenamiento del Modelo spaCy

Para ejecutar el entrenamiento completo del modelo:

```
python -m spacy train ./entrenamiento/spacy/config.cfg \
  --paths.train ./entrenamiento/spacy/train.spacy \
  --paths.dev ./entrenamiento/spacy/dev.spacy \
  --output ./entrenamiento/spacy/modelo
```

Métricas clave y objetivos

Métrica	Objetivo	Descripción
CATS_SCORE	> 0.90	Macro Accuracy / F1 para categorización de intenciones.
SCORE	> 0.90	Puntuación combinada del pipeline.
LOSS TEXTCAT	→ 0.00	Pérdida en la clasificación multiclase.

Estructura de artefactos resultantes en entrenamiento/spacy/modelo/:

- model-best/: Mejor modelo alcanzado (mayor CATS_SCORE), seleccionado para producción.
- model-last/: Último checkpoint del proceso de optimización.

6. Configuración LLM (Ollama)

Cada intención se vincula a un modelo especializado mediante Modelfiles específicos:

Intención	Modelo Ollama	Propósito
compra	chatbot-compra / chatbot-{dominio}-compra	Gestión de ventas, cualificación y presupuestos.
consulta_tecnica	chatbot-consulta / chatbot-{dominio}-consulta	Soporte técnico, diagnóstico y resolución.
otro	chatbot-otro / chatbot-{dominio}-otro	Saludos, cortesía y respuestas fuera de catálogo.
Resúmenes	llama3.2 (base)	Generación de

Intención	Modelo Ollama	Propósito
		resúmenes estructurados de compra.

Gestión de Modelfiles

Comandos de generación y compilación en Ollama:

```
# Generar ficheros de configuración
python manage.py generar_modelfiles

# Generar y compilar los modelos directamente en Ollama
python manage.py generar_modelfiles --crear
```

7. Flujo Completo de Reentrenamiento

Guía de pasos consolidados para un ciclo de actualización integral:

```
# 1. Editar/añadir ejemplos en CSV
nano entrenamiento/datos/nlp.csv

# 2. Recargar en base de datos
python manage.py cargar_nlp --limpiar

# 3. Generar archivos binarios .spacy
python manage.py generar_nlp

# 4. Entrenar el modelo spaCy
python -m spacy train ./entrenamiento/spacy/config.cfg \
  --paths.train ./entrenamiento/spacy/train.spacy \
  --paths.dev ./entrenamiento/spacy/dev.spacy \
  --output ./entrenamiento/spacy/modelo

# 5. Regenerar y compilar modelos LLM
python manage.py generar_modelfiles --crear

# 6. Ejecutar test de verificación
python verificar_nlp.py entrenamiento/spacy/modelo/model-best
```

Despliegue automatizado con Docker

```
# Forzar reentrenamiento completo (recarga y compilación)
docker compose run --rm -e FORCE_TRAIN=1 -e FORCE_RELOAD=1 setup

# Solo reentrenar spaCy NLP (conservando datos existentes)
docker compose run --rm -e FORCE_TRAIN=1 setup
```

8. Buenas Prácticas y Troubleshooting

Promoción de ejemplos desde conversaciones reales

1. Acceder al panel de administración en /admin/entrenamiento/analysis/.
2. Filtrar por la intención detectada/corregida.
3. Seleccionar la acción 'Promover a ejemplo de entrenamiento' (crea EjemploNLP con origen='chatbot').

Matriz de buenas prácticas

Práctica Recomendada	Práctica a Evitar
Variar frases ('quiero', 'necesito', 'me gustaría')	Duplicar ejemplos idénticos repetidamente
Incluir entidades reales con límites exactos	Anotar entidades artificiales o inexistentes
Balancear clases (~50 ejemplos por intención)	Desbalance severo (e.g. 500 de una y 5 de otra)
Utilizar lenguaje natural real de usuarios	Utilizar lenguaje técnico excesivamente artificial

Resolución de problemas (Troubleshooting)

Error / Síntoma	Causa Probable	Solución
spans not aligned	Offsets no coinciden con tokens	Revisar delimitadores de entidades en el CSV
no examples for intent	Clase sin muestras en train/dev	Añadir más ejemplos o reducir dev-ratio
CUDA out of memory	Memoria VRAM de GPU saturada	Forzar uso de CPU: spacy train ... --gpu-id -1
Score bajo (< 0.85)	Datos insuficientes o ruidosos	Limpiar dataset, añadir ejemplos y más epochs
model-best no existe	Entrenamiento interrumpido	Revisar logs y aumentar max_steps / patience